Développement d'un site web dynamique (UAA12)

Exercices : les tableaux

Objets d'apprentissage :

- ✔ Construire une page WEB dynamique à l'aide du langage PHP.

Exercice 1 : Les Simpson

Dans cet exercice, tu vas afficher le contenu d'un tableau 2D sous la forme d'une liste de liste (en HTML).

Etape 1

Complète la définition du tableau PHP donné afin de sauvegarder les informations suivantes :

SIMPSON	pere	Homer
	mere	Marge
FLANDERS	pere	Ned
	mere	Maude
VAN HOUTEN	pere	Kirk
	mere	Luann

```php
<?php
$familles = [
  'Simpson' => [
    'pere' => 'Homer',
    'mere' => 'Marge'],
    ...
];
?>
```

Etape 2

Modifie la définition du tableau PHP pour ajouter la famille « Wiggum », dont le père se prénomme « Clancy » et la mère « Sarah ». Tu ne dois normalement pas modifier le reste de ton script pour que l'affichage soit correct !

Etape 3

Modifie la définition du tableau pour ajouter les infos sur les enfants de chacune des familles afin. Tu dois obtenir le résultats suivant :

SIMPSON	pere	Homer
	mere	Marge
	enfant1	Bart
	enfant2	Lisa
	enfant3	Maggie
FLANDERS	pere	Ned
	mere	Maude
	enfant1	Rod
	enfant2	Todd
VAN HOUTEN	pere	Kirk
	Mère	Luann
	enfant1	Milhouse
WIGGUM	pere	Clancy
	mere	Sarah
	enfant1	Ralph

Au niveau de l'encodage des enfants, la structure proposée n'est pas très efficace… *pourquoi* ? Tente de répondre à la question avant de passer à l'étape suivante !

Etape 4

La famille Spuckler est l'une des plus emblématiques et prolifiques de Les Simpson.

Composée de Cletus et Brandine Spuckler, un couple stéréotypé vivant dans une caravane délabrée à Springfield, elle compte une ribambelle d'enfants entre 27 et 44 selon les épisodes.

La liste des enfants connus est disponible sur cette page :
https://simpsons.fandom.com/fr/wiki/Famille_Spuckler

Jusqu'à présent, pour représenter une famille comme les Simpson ou les Flanders avec un petit nombre d'enfants (2 ou 3), on utilisait souvent une structure de tableau PHP avec des clés numériques ou textuelles comme `"enfant1"`, `"enfant2"`, etc.

Cette approche fonctionne pour des familles petites, mais devient **totalement inefficace** avec la famille Spuckler ! Créer des clés `"enfant1"` à `"enfant44"` est laborieux, difficile à parcourir, et ingérable si le nombre change.

Il est donc nécessaire de modifier la structure de notre tableau de la manière suivante :

```php
<?php
$familles = [
    'Simpson' => [
        'pere' => 'Homer',
        'mere' => 'Marge',
        'enfants' => ['Bart', 'Lisa', 'Maggie']
    ],
    'Flanders' => [
        'pere' => 'Ned',
        'mere' => 'Maude',
        'enfants' => ['Rod', 'Todd']
    ],
    'Van Houten' => [
        'pere' => 'Kirk',
        'mere' => 'Luann',
        'enfants' => ['Milhouse']
    ],
    'Wiggum' => [
        'pere' => 'Clancy',
        'mere' => 'Sarah',
        'enfants' => ['Ralph']
    ]
];
?>
```

Complète cette définition afin de sauvegarder les cinq premiers enfants des Spuckler.

Etape 5

```php
<?php
$familles = [
    'Simpson' => [
        'pere' => 'Homer',
        'mere' => 'Marge',
        'enfants' => ['Bart', 'Lisa', 'Maggie']
    ],
    'Flanders' => [
        'pere' => 'Ned',
        'mere' => 'Maude',
        'enfants' => ['Rod', 'Todd']
    ],
    'Van Houten' => [
        'pere' => 'Kirk',
        'mere' => 'Luann',
        'enfants' => ['Milhouse']
    ],
    'Wiggum' => [
        'pere' => 'Clancy',
        'mere' => 'Sarah',
        'enfants' => ['Ralph']
    ],
    'Spuckler' => [
        'pere' => 'Cletus',
        'mere' => 'Brandine',
        'enfants' => ['Inceste', 'Cody', 'Scout', 'Tiffany', 'Dylan']
    ]
];
?>
```

L'avantage de cette nouvelle structure est qu'elle facilite grandement les différentes opérations possibles sur les tableaux.

Essaie chaque opération suivante. *Ne copie/colle pas bêtement !* Essaie d'anticiper le résultat avant d'exécuter chaque instruction (il est parfois nécessaire d'utiliser `print_r($familles)`)

- **Ajout** d'une nouvelle famille

```php
$familles['Burns'] = [
 'pere' => 'Clifford',
 'mere' => 'Daphné',
 'enfants' => ['Charles']
];
```

- **Ajout** d'un nouvel enfant

```php
$familles['Spuckler']['enfants'][] = 'Hunter';
```

- **Parcours** des enfants d'une famille en particulier

```php
foreach($familles['Spuckler']['enfants'] as $enfant) {

    echo $enfant;

}
```

- **Parcours** des enfants de toutes les familles

```php
foreach ($familles as $nom => $famille) {
```

```
    echo $nom . "\n";
    foreach ($famille['enfants'] as $enfant) {
        echo "  - " . $enfant . "\n";
    }
}
```

- **Parcours** des enfants de toutes les familles (variante)

```
foreach ($familles as $nom => $famille) {
    echo $nom . "\n";
    foreach ($famille['enfants'] as $position => $enfant) {
        echo "  " . ($position + 1) . ". " . $enfant . "\n";
    }
}
```

- **Comptage**

```
var_dump(count($familles['Spuckler']['enfants']));
```

- **Recherche** :

```
var_dump(in_array('Mary', $familles['Spuckler']['enfants']));
var_dump(array_search('Mary', $familles['Spuckler']['enfants']));
```

- **Suppression** d'un enfant

```
$index = array_search('Bart', $familles['Simpson']['enfants']);
unset($familles['Simpson']['enfants'][$index]);
$familles['Simpson']['enfants'] = array_values($familles['Simpson']
['enfants']);
```

Etape 6

La famille Bouvier est une branche maternelle étendue des Simpson. Selma Bouvier (la sœur de Marge Simpson) adopte Ling mais **aucun père n'est connu** ni biologique, ni légal.

Sachant que l'on considère, dans notre structure de données, la famille Bouvier comme une famille « indépendante » de la famille Simpson (c'est-à-dire que l'on ne souhaite pas retenir le fait que Selma est la sœur de Marge), comment modifierais-tu la déclaration de `$familles` ?

Réponds à la question avant de passer à l'étape suivante.

Etape 7

Voici la structure mise à jour :

```php
<?php
$familles = [
    'Simpson' => [
        'pere' => 'Homer',
        'mere' => 'Marge',
        'enfants' => ['Bart', 'Lisa', 'Maggie']
    ],
    'Flanders' => [
        'pere' => 'Ned',
        'mere' => 'Maude',
        'enfants' => ['Rod', 'Todd']
    ],
    'Van Houten' => [
        'pere' => 'Kirk',
        'mere' => 'Luann',
        'enfants' => ['Milhouse']
    ],
    'Wiggum' => [
        'pere' => 'Clancy',
        'mere' => 'Sarah',
        'enfants' => ['Ralph']
    ],
    'Spuckler' => [
        'pere' => 'Cletus',
        'mere' => 'Brandine',
        'enfants' => ['Inceste', 'Cody', 'Scout', 'Tiffany', 'Dylan']
    ],
    'Bouvier' => [
        'mere' => 'Selma',
        'enfants' => ['Ling']
    ]
];
?>
```

On remarque que l'absence de la clé « pere » reflète mieux la réalité : le père n'existe tout simplement pas dans les données. C'est là le grand avantage des tableaux en PHP : leurs structures peuvent être hétérogènes et permettent donc de gérer énormément de cas spécifiques.

Cependant, cette grande permissivité apporte aussi un inconvénient considérable. Pour le comprendre, écris un petit script qui affiche la liste de tous les pères de toutes les familles (Homer, Ned, etc.) et observe le résultat.

Le script se trouve à l'étape suivante. *Mais essaie de le faire toi-même avant de regarder la réponse* !

Etape 8

Voici le script permettant d'afficher la liste des pères :

```php
foreach ($familles as $nom => $famille) {
    echo "$nom : Père = " . $famille['pere'] . "\n";
}
```

Ce script devrait produire l'erreur suivante :

```
Warning: Undefined array key "pere"
```

Cette erreur apparaît quand ton code essaie d'accéder à une clé qui n'existe pas du tout dans le tableau, comme `$famille['pere']` pour la famille Bouvier.

Comment pourrais-tu utiliser `isset()` pour corriger le problème et afficher « Père inconnu » lorsque le père d'une famille n'est pas connu (la réponse se trouve à l'étape suivante) ?

Etape 9

Voici le script qui utilise `isset()` pour éviter l'erreur :

```php
foreach ($familles as $nom => $famille) {
    if (isset($famille['pere'])) {
        echo "$nom : Père = " . $famille['pere'] . "\n";
    } else {
        echo "$nom : Père inconnu ❌\n";
    }
}
```

Exercice 2 : pays et capitales

Voici la définition d'un tableau associatif reprenant la liste de différents pays associés à leur capitale :

```php
<?php
$pays = array(
    "Italie" => "Rome",
    "Luxembourg" => "Luxembourg",
    "Belgique" => "Bruxelles",
    "Danemark" => "Copenhague",
    "Finlande" => "Helsinki",
    "France" => "Paris");
?>
```

Etape 1

Sur base de ce tableau, crée un script PHP qui affiche la capitale et le nom du pays. Fais en sorte que les pays s'affichent en gras.

Exemple de sortie :

```
Italie, Rome
Luxembourg, Luxembourg
Belgique, Bruxelles
```

Etape 2

Modifie ensuite la définition de `$pays` afin que l'on puisse retrouver, pour chaque pays, sa capitale ainsi que sa superficie en km² (fais quelques recherches sur internet pour trouver les valeurs).

Tu devras peut être adapter la structure du tableau !

Ensuite, reprends ton script créé à l'étape 1 et adapte-le afin que l'affichage propose également la superficie de chaque pays.

Exemple de sortie :

```
Italie (superficie : 302073 km²), Rome
Luxembourg (superficie : 2586 km²), Luxembourg
Belgique (superficie : 30688 km²), Bruxelles
```

Exercice 3 : des fruits

Le site php.net fournit une large documentation sur le langage PHP, notamment sur les différentes fonctions qui peuvent être utilisées avec les tableaux. Ainsi, pour chaque fonction, on retrouve :

- la description de la fonction (*ce qu'elle fait*) ;
- les entrées de la fonction (*ce dont elle a besoin pour fonctionner*) ;
- les sorties de la fonction (*ce qu'elle retourne comme résultat*) ;
- quelques exemples d'utilisation (*très utile pour comprendre rapidement !*).

Dans cet exercice, tu vas découvrir par toi-même quelques fonctions bien pratiques pour la manipulation de tableaux. Renseigne-toi sur les fonctions ci-dessous en utilisant la documentation en ligne :

Fonction	Documentation
`array_push`	https://www.php.net/manual/fr/function.array-push.php
`array_pop`	https://www.php.net/manual/fr/function.array-pop.php
`array_shift`	https://www.php.net/manual/fr/function.array-shift.php
`array_unshit`	https://www.php.net/manual/fr/function.array-unshift.php
`in_array`	https://www.php.net/manual/fr/function.in-array.php
`array_search`	https://www.php.net/manual/fr/function.array-search.php
`unset`	https://www.php.net/manual/fr/function.unset.php
`array_unique`	https://www.php.net/manual/fr/function.array-unique.php
`sort`	https://www.php.net/manual/fr/function.sort.php
`array_merge`	https://www.php.net/manual/fr/function.array-merge.php

Note : d'autres exemples sont disponibles sur le site w3schools.com (en anglais). Sur base du tableau suivant :

```php
<?php
  $fruits = array("pomme", "poire", "orange", "mangue", "banane");
?>
```

Ecris les instructions réalisant les actions suivantes (bien évidemment, en utilisant les fonctions listée à la page précédente) :

1. Supprime le dernier fruit du tableau (*une fonction à utiliser*)
2. Ajoute un nouveau fruit de ton choix à la fin du tableau (*une fonction à utiliser*)
3. Ajoute un nouveau fruit de ton choix au début du tableau (*une fonction à utiliser*)
4. Supprime le premier fruit du tableau (*une fonction à utiliser*)
5. Affiche « La courgette est un fruit » ou « La courgette n'est pas un fruit » selon si la courgette se trouve dans le tableau ou non (*une fonction à utiliser*)
6. Supprime l'orange du tableau (*trois fonctions à utiliser*)

Après avoir supprimé l'orange, affiche ton tableau avec `print_r`. Que remarques-tu au niveau des clés ?

Utilise l'instruction suivante pour ré-indexer les clés :

```php
<?php
  $fruits = array_values($fruits);
?>
```

Exercice 4 : drill

Situation 1

```php
$fruits = ["pomme", "banane", "orange"];
```

Ajouter « kiwi » puis afficher le tableau.

Résultat attendu :

```
[0] => pomme
[1] => banane
[2] => orange
[3] => kiwi
```

Situation 2

```php
$joueurs = ["Alice", "Bob", "Charlie", "Daria"];
```

À faire : Supprimer Bob (index 1) et réindexer le tableau.

Résultat attendu :

```
[0] => Alice
[1] => Charlie
[2] => Daria
```

Situation 3

```php
$notes = ["math" => 18, "physique" => 14, "chimie" => 16];
```

À faire : Afficher chaque matière et sa note sur une ligne séparée.

Résultat attendu :

```
math : 18
physique : 14
chimie : 16
```

Situation 4

```php
$eleve = ["prenom" => "Tom", "age" => 19, "filière" => "DUT"];
```

À faire : Vérifier si la clé « email » existe. Afficher « Pas d'email » si elle est absente.

Résultat attendu : pas d'email

Situation 5

```php
$scores = [45, 72, 38, 91, 60, 28, 85];
```

À faire : Compter combien de scores sont strictement supérieurs à 50. Afficher le nombre.

Résultat attendu : 4

Situation 6

```php
$produits = ["stylo", "cahier", "gomme", "règle", "colle"];
```

À faire : Afficher l'index de « règle » dans le tableau.

Résultat attendu : 3

Situation 7

```php
$tags = ["php", "web", "php", "sql", "web", "php"];
```

À faire : Supprimer les doublons et réindexer le tableau.

Résultat attendu :

```
[0] => php
[1] => web
[2] => sql
```

Situation 8

```php
$villes = ["Lyon", "Paris", "Bordeaux", "Marseille"];
```

À faire : Trier le tableau par ordre alphabétique et l'afficher.

Résultat attendu :

```
[0] => Bordeaux
[1] => Lyon
[2] => Marseille
[3] => Paris
```

Situation 9

```php
$matins = ["café", "toast"];
$midis = ["salade", "soupe", "pain"];
```

À faire : Fusionner les deux tableaux en un seul et l'afficher.

```
[0] => café
[1] => toast
[2] => salade
[3] => soupe
[4] => pain
```

Situation 10

```php
$eleves = [
    ["nom" => "Alice", "age" => 20, "note" => 17],
    ["nom" => "Bob",   "age" => 21, "note" => 14],
    ["nom" => "Clara","age" => 19, "note" => 19],
];
```

À faire : Extraire uniquement les noms dans un tableau et l'afficher.

Résultat attendu :

```
[0] => Alice
[1] => Bob
[2] => Clara
```

Exercice 5 : fonctions et tableaux (dépassement)

Tu travailles sur un projet de gestion d'étudiants. Chaque étudiant est représenté par un tableau associatif contenant les informations suivantes : nom, prénom, âge et note moyenne.

Pour chaque étudiant, on lui associe une clé ayant la forme suivante : « `<nom>_<prenom>` ». Par exemple, « Jules Dupont » sera associé à la clé « `Dupont_Jules` ».

Etape 1

Déclare un tableau vide et stocke-le dans une variable globale nommée `$listeEtudiants`.

Etape 2

Crée une fonction `ajouterEtudiant($etudiant)` qui ajoute un nouvel étudiant dans le tableau. Deux cas sont à prévoir :

(1) Si la combinaison `<nom>_<prenom>` existe déjà, le programme affichera le message d'erreur « Echec de l'ajout. Un étudiant avec la même clé existe déjà »

(2) Sinon, le programme affichera « L'étudiant a été ajouté avec succès »

Etape 3

Crée la fonction `afficherEtudiants()` qui affiche la liste des étudiants avec toutes les informations disponibles.

Etape 4

Crée la fonction `rechercherEtudiant($nom, $prenom)` qui se charge de rechercher un étudiant dans le tableau sur base de son nom et prénom reçu en entrée. Deux cas sont à prévoir :

(1) Si l'étudiant est trouvé, la fonction affichera toutes les informations disponibles sur lui

(2) Sinon, le programme affichera « Etudiant non trouvé »



Pour implémenter cette fonction, il n'est pas forcément nécessaire de parcourir le tableau avec une boucle. On peut très bien tester simplement la présence ou non de la clé !

Etape 5

Déclare un tableau d'étudiants et teste les trois méthodes définies précédemment.

Références

Les présents exercices ont été élaborés à l'aide des ressources suivantes :

- Forbes, A. (2013). The Joy of PHP: A Beginner's Guide to Programming Interactive Web Sites.
- Vaswani, V. (2021). PHP A Beginner's Guide.
- Cours de « Développement d'applications WEB » (2018), Hénallux